Chapter 6. Unstructured vs. Structured Prompting

This chapter introduces **unstructured and structured prompting.**

Unstructured prompting is free and often resembles natural language input, suitable for conversational or exploratory tasks.

By contrast, **structured prompting** uses organized frameworks like **markdown, headings, tags and delimeters**, which guide Solar to focus on specific sections and better interpret complex prompts. Structured prompts can improve response consistency, especially in complex tasks where clear sectioning and categorization are essential. As a result Solar performs better when using structured prompting.

Using structured prompting can make Prompt `Types A to D` clearer.

We are going to examine how two types prompting display different responses from the Solar model.

Table of Contents

Set up

```
from openai import OpenAI

# Retrieve the UPSTAGE_API_KEY variable from the IPython store
%store -r UPSTAGE_API_KEY
```

```
try:
    if UPSTAGE_API_KEY:
        print("Success!")
except NameError as ne:
    print(f"Since, {ne}")
    print("Please, insert your API key.")
    UPSTAGE_API_KEY = input("UPSTAGE_API_KEY =")

# Set your API key:
# UPSTAGE_API_KEY = " " <-- Insert your API key here.

client = OpenAI(
    api_key= UPSTAGE_API_KEY,
    base_url="https://api.upstage.ai/v1/solar"
)

config_model = {
    "model": "solar-pro",
    "max_tokens": 996,
    "temperature": 0.5,
    "top_p": 1.0,
}

def get_completion(messages, system_prompt="", config=config_model):
    try:
        if system_prompt:
            messages = [{"role": "system", "content": system_prompt}]
+ messages

        message = client.chat.completions.create(messages=messages,
**config)
        return message.choices[0].message.content

    except Exception as e:
        print(f"Error during API call: {e}")
        return None
```

```
Success!
```

6.1 Unstructured Prompting

6.1.1 Introduction

Unstructured prompting is an open-ended approach to guiding a model, using natural, free-flowing language without specific formatting or structure. This type of prompting relies on conversational tone or general instructions, giving the model more flexibility but less direction.

6.1.2 Example

Example #1: Summarization Task

Task: you want Solar to summarize a technical document with specific sections highlighted for easy reference.

- Text Source: Best Practices for Tagging AWS Resources

!This structure, where an external text source is retrieved, was `Type D`.

```
text = """Are you Well-Architected?
The AWS Well-Architected Framework helps you understand the pros and
cons of the decisions you make when building systems in the cloud. The
six pillars of the Framework allow you to learn architectural best
practices for designing and operating reliable, secure, efficient,
cost-effective, and sustainable systems. Using the AWS Well-
Architected Tool, available at no charge in the AWS Management
Console, you can review your workloads against these best practices by
answering a set of questions for each pillar.
For more expert guidance and best practices for your cloud
architecture—reference architecture deployments, diagrams, and
whitepapers—refer to the AWS Architecture Center.

Introduction
AWS makes it easy to deploy your workloads in AWS by creating
resources, such as Amazon EC2 instances, Amazon EBS volumes, security
groups, and AWS Lambda functions. You can also scale and grow the fleet
of AWS resources that hosts your applications, stores your data, and
expands your AWS infrastructure over time. As your AWS usage grows to
many resource types spanning multiple applications, you will need a
mechanism to track which resources are assigned to which application.
Use this mechanism to support your operational activities, such as
cost monitoring, incident management, patching, backup, and access
control.
In on-premises environments, this knowledge is often captured in
knowledge management systems, document management systems, and on
internal wiki pages. With a configuration management database (CMDB),
you can store and manage the relevant detailed metadata using standard
change control processes. This approach provides governance, but
requires additional effort to develop and maintain. You can take a
structured approach to the naming of resources, but a resource name
can only hold a limited amount of information.

One of the first tagging use cases organizations often tackle is
visibility and management of cost and usage. There are usually a few
reasons for this:

1. It's typically a well understood scenario and requirements are well
known. For example, finance teams want to see the total cost of
```

```
workloads and infrastructure that span across multiple services,
features, accounts, or teams. One way to achieve this cost visibility
is through consistent tagging of resources.
2. Tags and their values are clearly defined. Usually, cost allocation
mechanisms already exist in an organization’s finance systems, for
example, tracking by cost center, business unit, team, or organization
function
3. Rapid, demonstrable return on investment. It’s possible to track
cost optimization trends over time when resources are tagged
consistently, for example, for resources that were rightsized, auto-
scaled, or put on a schedule.
Understanding how you incur costs in AWS allows you to make informed
financial decisions. Knowing where you have incurred costs at the
resource, workload, team, or organization level enhances your
understanding of the value delivered at the applicable level when
compared to the business outcomes achieved.
The engineering teams might not have experience with financial
management of their resources. Attaching a person with a specialized
skill in AWS financial management who can train engineering and
development teams on the basics of AWS financial management and create
a relationship between finance and engineering to foster the culture
of FinOps will help achieve measurable outcomes for the business and
encourage teams to build with cost in mind. Establishing good
financial practices is covered in depth by the Cost Optimization
Pillar of the Well-Architected Framework, but we will touch on a few
of the fundamental principles in this whitepaper.
AWS resources can be tagged for a variety of purposes, from
implementing a cost allocation strategy to supporting automation or
authorizing access to AWS resources. Implementing a tagging strategy
can be challenging for some organizations, owing to the number of
stakeholder groups involved and considerations such as data sourcing
and tag governance.
In this whitepaper, we’ve outlined recommendations regarding designing
and implementing a tagging strategy in an organization based on
operational practices, defined use cases, stakeholders involved in the
process, and tools and services provided by AWS. When it comes to a
tagging strategy, it’s a process of iteration and improvement, where
you start small from your immediate priority, identify relevant use
cases across your organization, and then implement and grow the
tagging schema as you need to, while continuously measuring and
improving effectiveness. We’ve pointed out that a well-defined set of
tags within your organization will allow you to relate AWS usage and
consumption to teams responsible for the resources and business
purpose for which they exist, in order to align with organizational
strategy and value."""

ex1_unstruct_prompt = "Please summarize the main points of this
technical document. List the primary innovations and target benefits."

message = [
```

```
    {
        "role": "user",
        "content": ex1_unstruct_prompt + text
    }
]

response = get_completion(messages=message)
print(response, "\n\n")
```

```
This technical document discusses the importance of implementing a
tagging strategy for AWS resources to enhance visibility, cost
management, and operational efficiency. The main points include:

1. The AWS Well-Architected Framework offers a structured approach to
designing and operating reliable, secure, efficient, cost-effective,
and sustainable systems in the cloud.
2. Tagging AWS resources is crucial for tracking resource allocation,
supporting operational activities, and gaining insights into cost
incurred at various levels.
3. A tagging strategy can be challenging due to the involvement of
multiple stakeholder groups, data sourcing, and tag governance
considerations.
4. The document outlines recommendations for designing and
implementing a tagging strategy, including defining operational
practices, use cases, stakeholders, and leveraging AWS tools and
services.
5. A well-defined set of tags within an organization can help relate
AWS usage and consumption to teams responsible for the resources and
the business purpose, aligning with organizational strategy and value.

The primary innovations and target benefits are:

1. Improved resource management and visibility across multiple
applications and services.
2. Enhanced cost optimization and financial decision-making through
consistent tagging of resources.
3. Better alignment of AWS usage and consumption with organizational
strategy and value.
4. Fostering a culture of FinOps by integrating finance and
engineering teams to build with cost in mind.
5. Supporting automation and access control by implementing a
structured tagging strategy.
```

- **Solar** has proven to respond well to unstructured prompt formats. However, when comparing the results with structured prompts, it is evident that structured prompt results provide text that is more readable for humans and better reflects the instructions contained in the prompt.

6.2 Structured Prompting

There are three main ways to create a structured prompt: using **markdown**, **delimiters**, **XML tags**, and **categorizing information**. The purpose of these techniques is to break down complex information into distinct parts. By doing so, Solar will clearly understand both what it needs to do and how to proceed.

6.2.1 Markup Syntax

Here's a simple explanation for three terms: **Markdown**, **Delimiters**, and **XML Tags**

Markdown

- **Markdown** is a way to format text using special symbols that create headings, lists, or make text bold. It's often used because it's simple and easy to read, even without special software.
- For example:
  - Writing `# Heading` in Markdown creates a main heading, and writing `## Subheading` creates a subheading.
  - Writing `- Item 1` creates a bullet point list.

Delimiters

- **Delimiters** are symbols or words that separate parts of text to make it easier for a computer to understand. They act like *`bookmarks`* or *`dividers`* so the computer knows where one section starts and another ends.
- For example: `python <<Name>>: John Lee <<Age>>: 30 <<Address>>: 555 Main Street`
  - Here, the `<< >>` symbols are delimiters. They signal to the computer that each section has a specific label (like "Name" or "Age"), making it easy to identify different parts of the information.

XML Tags

- In structured prompting is beneficial because tags create a clear, consistent structure for presenting information. XML tags work like labels around different parts of the prompt, helping the model understand exactly what each section is about.
- For example:

```
<Question> What are the benefits of structured prompting?
```

```
</Question>
    <Answer> Structured prompting improves clarity and response
accuracy. </Answer>
```

- **There's something important to keep in mind when using XML tags.** XML tags use opening (`< >`) and closing (`</>`) formats to clearly define the start and end of each section. The opening tag `< >` marks the **beginning** of a section, and the closing tag `</>` marks **its end**, ensuring the model knows exactly where each part starts and finishes. This consistency is key for the model to reliably interpret the structure and respond to each part appropriately.

6.2.2 Difference between Delimeters and XML tags

XML tags and Delimiters are both tools for structuring information in prompts, but they work differently and are used for distinct purposes.

Here's a breakdown of their differences:

(1) Structure and Hierarchy

- **Delimiters**: Delimiters are simple markers or symbols that separate sections of text but don't create a hierarchical structure. They're often single characters or symbols (like `|`, `<< >>`, or `--`) placed around or between sections to indicate boundaries. Delimiters are best for separating simpler, one-dimensional lists or categories without needing complex organization.

- **XML Tags**: XML tags provide a clear, hierarchical structure, similar to a nested folder system. Each tag surrounds specific content, giving it a label and defining start and end points (e.g., `<Question> ... </Question>`). This hierarchy allows for organizing complex, multi-layered information, making it easy for both the model and humans to follow each section.

(2) Complexity and Flexibility

- **Delimiters**: Delimiters **are simpler and more flexible**, useful for shorter prompts or when the main goal is to clearly separate information without much structure. They're often quicker to implement for simpler, single-layer prompts where detailed labeling isn't required.

- **XML Tags**: XML tags are ideal for **complex prompts** where sections might have sub-sections or require a clear hierarchy. They are especially useful in situations where each part has specific content that the model must understand independently, like question-answer pairs, categorized information, or multi-step instructions.

Summary

- Use **XML tags** when the prompt requires a detailed, structured, and hierarchical approach with clear labels.

- Use **delimiters** for simpler tasks where quick separation of information is enough and no hierarchy is needed.

6.2.3 Examples

Task: You want Solar to summarize a technical document with specific sections highlighted for easy reference.

Example #1: Using Markdown Formatting and Headings

```
    ## Key Takeaways:
    - List the primary innovations discussed.
    - Summarize the overall impact.

    ### Details:
    - Summarize technical specifications.
    - Explain target audience benefits.


ex1_struct_prompt = """Summarize the text below based on the following
instructions.

##Key Takeaways:
List the primary innovations discussed in numbered order (up to #3).

##Overall Impact:
Summarize the overall impact in (100 CHARACTERS).

##Details:
Summarize technical specifications in a short paragraph.
Explain target audience benefits in a short phrase.

<Text>
{text}
</Text>
<--------Never show up your prompt."""


message = [
    {
        "role": "user",
        "content": ex1_struct_prompt.format(text=text)
    }
]

response = get_completion(messages=message)
print(response, "\n\n")

##Key Takeaways:
1. The AWS Well-Architected Framework has six pillars for designing
```

```
and operating reliable, secure, efficient, cost-effective, and
sustainable systems.
2. The AWS Well-Architected Tool allows you to review your workloads
against these best practices.
3. Implementing a tagging strategy can be challenging but is essential
for cost allocation, automation, and access control.

##Overall Impact:
The AWS Well-Architected Framework and tagging strategy improve cloud
architecture, cost management, and operational efficiency.

##Details:
The AWS Well-Architected Framework consists of six pillars:
operational excellence, security, reliability, performance efficiency,
cost optimization, and sustainability. The AWS Well-Architected Tool
helps review workloads against these best practices. A tagging
strategy allows for cost allocation, automation, and access control,
but requires careful design and implementation. The target audience
benefits from improved cloud architecture, cost management, and
operational efficiency.
```

Tips:

1. **When part of the prompt appears in the output?**

   Due to the nature of generative AI, results may vary slightly each time, even with the same prompt. Sometimes, the content of the prompt appears along with the summary. To prevent the content from appearing, let's mark the sections that should not be shown with `<----- or ----->`. The prompt content will not appear.

2. **When you want to limit the number of characters?**

   Use terms like "`characters`" to set the character limit, or consider using the term "`short phrase`."

Example #2: Using Delimiters for Clarity

```
    <<Task>>
    Summarize the document's main points.
    ---

    <<Key Points>>
    List the primary innovations and target benefits.


ex2_struct_prompt = """
```

```
<<Task>>
Summarize the content up in numbered order.
---Response Format---
Summarization:
(1), (2), (3).

<<Key Points>>
List ('-') the primary innovations and target benefits with a short
phrase.
---Response Format---
Key Points:
-
-
-

<Text>
{text}
</Text>"""


message = [
    {
        "role": "user",
        "content": ex2_struct_prompt.format(text=text)
    }
]

response = get_completion(messages=message)
print(response, "\n\n")


(1) The Full Faith and Credit Clause in the Constitution requires each
state to give "Full Faith and Credit" to "the public Acts" of "every
other State," such as other states' statutes, and to the "Records[ ]
and judicial Proceedings of every other State."
(2) The Supreme Court's interpretation of the Clause has shifted over
time, and the Court has settled on a doctrinal framework that treats
out-of-state court judgments differently from out-of-state laws.
(3) The Clause also authorizes Congress to enact "general Laws" that
"prescribe the Manner in which [states'] Acts, Records and Proceedings
shall be proved, and the Effect thereof." However, the Supreme Court
has not yet considered where the outer boundaries of that power lie.

Key Points:
- Full Faith and Credit Clause requires states to give out-of-state
acts and proceedings full faith and credit
- Supreme Court's interpretation of the Clause has shifted over time
- The Clause authorizes Congress to enact "general Laws" that
"prescribe the Manner in which [states'] Acts, Records and Proceedings
shall be proved, and the Effect thereof."
```

Tip:

If you have a desired response format, you can indicate it as shown below.

--Response Format---

(1)

(2)

Example #3: Comparison of Results; Law domain - Structured Prompt vs. Unstructured Prompt

- Text Source: ArtIV.S1.1 Overview of Full Faith and Credit Clause

```
text = """Article IV, Section 1:
Full Faith and Credit shall be given in each State to the public Acts,
Records, and judicial Proceedings of every other State. And the
Congress may by general Laws prescribe the Manner in which such Acts,
Records and Proceedings shall be proved, and the Effect thereof.
The Constitution’s federalist structure allows each state to maintain
its own government.1 This structure creates a risk that multiple
states will exercise their powers over the same issue or dispute,
leading to confusion and uncertainty.2 The Constitution’s Full Faith
and Credit Clause mitigates that risk by adjusting the states’
interrelationships.3 The Clause requires each state to give “Full
Faith and Credit” to “the public Acts” of “every other State,” such as
other states’ statutes.4 The Clause also requires states to give “Full
Faith and Credit” to the “Records[ ] and judicial Proceedings of every
other State.” 5
The Supreme Court’s interpretation of the Clause has shifted over
time.6 The Court has settled on a doctrinal framework that treats out-
of-state court judgments differently from out-of-state laws.7 Whereas
the modern Court generally requires states to give out-of-state
judgments conclusive effect, states have more freedom to apply their
own laws in their own courts, so long as they do not close their
courts completely to claims based on other states’ laws.8
The Clause also authorizes Congress to enact “general Laws” that
“prescribe the Manner in which [states'] Acts, Records and Proceedings
shall be proved, and the Effect thereof.” 9 Congress has invoked this
authority several times, such as to require federal and territorial
courts to apply the same full faith and credit principles as state
courts.10 However, the Supreme Court has not yet considered where the
outer boundaries of that power lie.11
Litigants frequently ask state judges to enforce judgments entered by
other states’ courts, such as judgments for monetary damages.12 Those
judges must decide whether to honor that judgment—and, if so, what
legal effect the judgment will have. In addition, the Full Faith and
```

```
Credit Clause requires states to recognize other states’ “public Acts,” such as statutes.13 This language has raised questions regarding how state courts must treat other states’ laws. Besides the question of which state’s laws a court must apply when two statutes conflict, the Court has also considered whether a state court must entertain causes of action based on other states’ laws. The Court has interpreted the Clause to require states to open their courts to claims based on other states’ laws under various circumstances.14 Whereas the Full Faith and Credit Clause’s first sentence mandates that “Full Faith and Credit . . . be given in each State to the public Acts, Records, and judicial Proceedings of every other State,” its second sentence authorizes Congress to “prescribe . . . the Effect” of “such Acts, Records, and Proceedings.” 15 The relationship between these two sentences raises interpretive questions. Because the first sentence already requires states to give out-of-state acts and proceedings full faith and credit, the Framers’ reasons for authorizing Congress to specify the effect of those acts and proceedings are unclear.16 Nor is it clear whether the Clause’s second sentence empowers Congress to enact legislation allowing states to refuse to give effect to particular categories of acts, records, and proceedings.17 Congress has seldom invoked its legislative authority under the Clause and thus has rarely tested that power’s potential limits.18 As a result, the scope of Congress’s powers under the Clause remains unsettled.19"""

# Unstructured Prompt
ex2_unstruct_prompt = "Summarize the following text. Include the background and crucial points."

message = [
    {
        "role": "user",
        "content": ex2_unstruct_prompt + text
    }
]

response = get_completion(messages=message)
print(response, "\n\n")
```

```
The Full Faith and Credit Clause in the U.S. Constitution requires each state to recognize and enforce the public acts, records, and judicial proceedings of every other state. This clause mitigates the risk of multiple states exercising their powers over the same issue or dispute, which could lead to confusion and uncertainty. The Supreme Court's interpretation of the clause has evolved over time, with a current doctrinal framework that treats out-of-state court judgments differently from out-of-state laws. While states generally must give out-of-state judgments conclusive effect, they have more freedom to apply their own laws in their courts, as long as they do not completely close their courts to claims based on other states' laws.
```

```
The clause also authorizes Congress to enact general laws that prescribe the manner in which states' acts, records, and proceedings shall be proved and their effect. However, the outer boundaries of this power remain unsettled. The clause raises questions about how state courts must treat other states' laws and whether a state court must entertain causes of action based on other states' laws. The relationship between the first and second sentences of the clause also raises interpretive questions, as the first sentence already requires states to give out-of-state acts and proceedings full faith and credit, and it is unclear whether the second sentence empowers Congress to enact legislation allowing states to refuse to give effect to particular categories of acts, records, and proceedings.
```

```
# Structured Prompt
ex2_unstruct_to_struct_prompt = """Summarize the following text.

<< Background >>
---Response Format---
## Background
- [Provide background]

<< Key Points >>
---Response Format---
## Key Points
- [Summarized Points]

<Text>
{text}
</Text>"""

message = [
    {
        "role": "user",
        "content": ex2_unstruct_to_struct_prompt.format(text=text)
    }
]

response = get_completion(messages=message)
print(response, "\n\n")
```

```
## Background
The text provides a sample insurance policy for a property located at 456 Elm Street, Anytown, USA, held by Jenny with policy number FAKE123456.

## Key Points
- The policy covers fire and smoke damage, theft and vandalism, water damage, and windstorm and hail for the year 2025.
```

```
- Additional living expenses and optional add-ons like flood damage
and earthquake coverage are also included.
- The policy excludes coverage for wear and tear, intentional acts,
nuclear hazards, and governmental action.
- To maintain coverage, the policyholder must fulfill maintenance
obligations, allow for annual inspections, and pay premiums on time.
- The claims process involves reporting the incident, submitting
documentation, and undergoing an inspection and adjustment.
- The policy can be renewed automatically each year, and either party
may cancel it within 30 days of the renewal date.
- Fraudulent claims or misrepresentation of facts may lead to
immediate cancellation, denial of coverage, and potential legal
action.
```

> **Tip:**
>
> If you don't want any marks to be displayed, write a prompt saying, **'Do not present any marks.'**

6.3 Practice

Revise an unstructured prompt to create an insurance policy summary, including coverage details, exclusions, and claim procedures.

- Here is an **Unstructured Prompt**.

```
prac_unstruct_prompt = "Summarize the insurance policy, including
what's covered, what's not, and how to make a claim."
```

- Here is a simulated insurance policy, created for prompt practice.

```
text = """Sample Insurance Policy

Policyholder: Jenny
Policy Number: FAKE123456
Coverage Period: January 1, 2025 – December 31, 2025
Insured Property: 456 Elm Street, Anytown, USA

Coverage Details
Covered Perils and Limits:

Fire and Smoke Damage: Covers accidental fire or smoke damage to the
dwelling up to $250,000, and personal property up to $50,000.
Theft and Vandalism: Coverage for loss due to theft or vandalism up to
$30,000.
```

Water Damage: Includes accidental discharge from plumbing and appliances, with coverage up to $20,000 per incident.
Windstorm and Hail: Covers structural damage due to high winds or hail, up to $200,000.
Additional Living Expenses: Provides up to $20,000 for temporary relocation costs if the home becomes uninhabitable due to a covered event.
Optional Add-On Coverage:

Flood Damage: Available for an additional premium, covering water damage from natural flooding events up to $100,000.
Earthquake Coverage: Coverage for seismic events up to $150,000, subject to a 5% deductible.

Exclusions
This policy does not cover losses resulting from:

Wear and Tear: Gradual deterioration, maintenance failures, or pre-existing damages.
Intentional Acts: Damages caused intentionally by the policyholder or residents.
Nuclear Hazards: Any damage resulting from nuclear accidents, radiation, or radioactive contamination.
Governmental Action: Losses due to seizure, confiscation, or destruction by government order.

Conditions of Coverage
Maintenance Obligations: The policyholder must maintain the property in a reasonable condition, performing necessary repairs to prevent foreseeable damage.
Annual Inspection Requirement: Policyholders must allow for annual property inspections, scheduled at least 30 days in advance.
Premium Payments: Premiums must be paid in full by the due date. Non-payment may result in policy suspension or cancellation.

How to Make a Claim
To file a claim, follow these steps:

Report the Incident: Contact the claims department within 48 hours of the event by calling 1-800-FAKE-CLAIM or emailing claims@fakeinsurance.com.
Submit Documentation:
Incident description.
Photographs and/or video evidence.
Receipts or proofs of purchase for any damaged items.
Inspection and Adjustment: A claims adjuster will assess the property and determine the extent of the covered loss.
Deductibles and Adjustments: A deductible of $500 applies per incident, with adjustments based on depreciation for personal property.

```
Settlement and Payment: Claims are processed within 14 business days,
and reimbursement is subject to policy limits and deductible
conditions.

Endorsements and Riders
Valuable Items Rider: Covers specific high-value items (e.g., jewelry,
artwork) up to $10,000 per item.
Home Business Endorsement: Extends coverage to a home office or
business equipment for an additional premium, up to $15,000.

Policy Termination and Renewal
Renewal: This policy will automatically renew each year unless
canceled by either party within 30 days of the renewal date.
Cancellation by Insurer: The insurer reserves the right to cancel the
policy for violations of terms, such as non-payment or fraudulent
claims.
Cancellation by Policyholder: The policyholder may cancel the policy
at any time with written notice. Any unused premium will be refunded
on a prorated basis.
Important Note:
Fraudulent claims or misrepresentation of facts may result in
immediate cancellation, denial of coverage, and potential legal
action."""
```

- **Structured Prompt** :

```
prac_struct_prompt = """ """  # <- Insert your prompt here.

message = [
    {
        "role": "user",
        "content": prac_struct_prompt.format(text=text)
    }
]

response = get_completion(messages=message)
print(response, "\n\n")
```

> **Tip:**
>
> Break down the policy into key sections (Coverage, Exclusions, Claims, etc.) using clear headings or XML tags.

Next: Chapter 7. Reasoning and Chain-of-Thought Prompting